1)import pandas as pd

Bring in the pandas library and call it pd so I can use it to work with data easily
(pandas ek data sambhalne ka dabba hai. Isko code mein bula lo aur 'pd' kehke kaam chala lo)

2)from sklearn.model_selection import train_test_split

From the big machine learning toolbox (sklearn), go into the model_selection section, and bring me the train_test_split tool so I can split my data for training and testing

3)from sklearn.ensemble import RandomForestClassifier

From the machine learning toolbox (sklearn), go to the 'ensemble' section and bring me the RandomForestClassifier tool so I can build a smart model that makes predictions using a forest of decision trees.

 # Step 1: Create a Simple Dataset

Part	Meaning
`data =`	You're storing this info in a variable called `data`
`{ ... }`	This is a **dictionary**, like a table with keys and values
`"column_name": [values]`	Each key is a column, and each list is that column's data

You are making **a** mini data table using a Python dictionary. Each symptom is a column, and each row is a person. You are preparing this data to use later for training a machine learning model.

💡 **Fun Tip:**
If you ever want to generate fake data for testing, you can use this:

```
from sklearn.datasets import make_classification
X, y = make_classification(n_samples=100, n_features=4)
```

This creates **100 rows** with **4 features (symptoms)** and a result (like disease yes/no).

Convert to DataFrame

```
df = pd.DataFrame(data)
```

Convert the rough data (dictionary format) into a clean, Excel-style table using pandas, and save it in a variable called df for easy use later

chest_pain	shortness_of_breath	fatigue	left_arm_pain	disease
1	1	1	1	heart_disease
1	0	1	0	heart_disease
0	1	0	0	flu
0	0	1	0	flu
1	1	0	1	heart_disease
0	0	1	0	flu
1	0	0	0	flu
0	1	1	0	heart_disease

It's now easier to **see**, **read**, and **use** the data.

Step 2: Prepare Data for Training

```
X = df.drop("disease", axis=1)  # Features (symptoms)
```

- Make a new table from df, but remove the **'disease'** column. Keep all the symptoms only.

```
y = df["disease"]  # Target (disease)
```

- Pick only the 'disease' column from the table. This is the label we want to predict.

Separate the data into two parts:
X = all symptoms (what we give the model)
y = the disease (what the model has to guess)

Split data into training and testing sets

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

Imagine you're training a robot to recognize diseases from symptoms
You give the robot 8 examples and say:
"Yeh dekho — jab chest pain + fatigue hoti hai, to heart disease hota hai. Jab chest pain nahi hoti, to flu hota hai."
Basically:
Train karo, seekho, yaad rakho.

[test_size=0.2 means:
➤ 80% data se model train hota hai
➤ 20% data se model test hota hai]

You hide 2 examples from the robot and ask:
"

Step 3: Train a Machine Learning Model

```
model = RandomForestClassifier()
model.fit(X_train, y_train)
```

.fit() ka matlab hota hai: "**Model ko training do, seekhne do**."

X_train → symptoms (input)
y_train → disease (output)

Ab model har row se ye seekhta hai:

"**Agar chest pain ho aur fatigue ho, to mostly heart disease hoti hai**..."

So now your model learns patterns from this data.

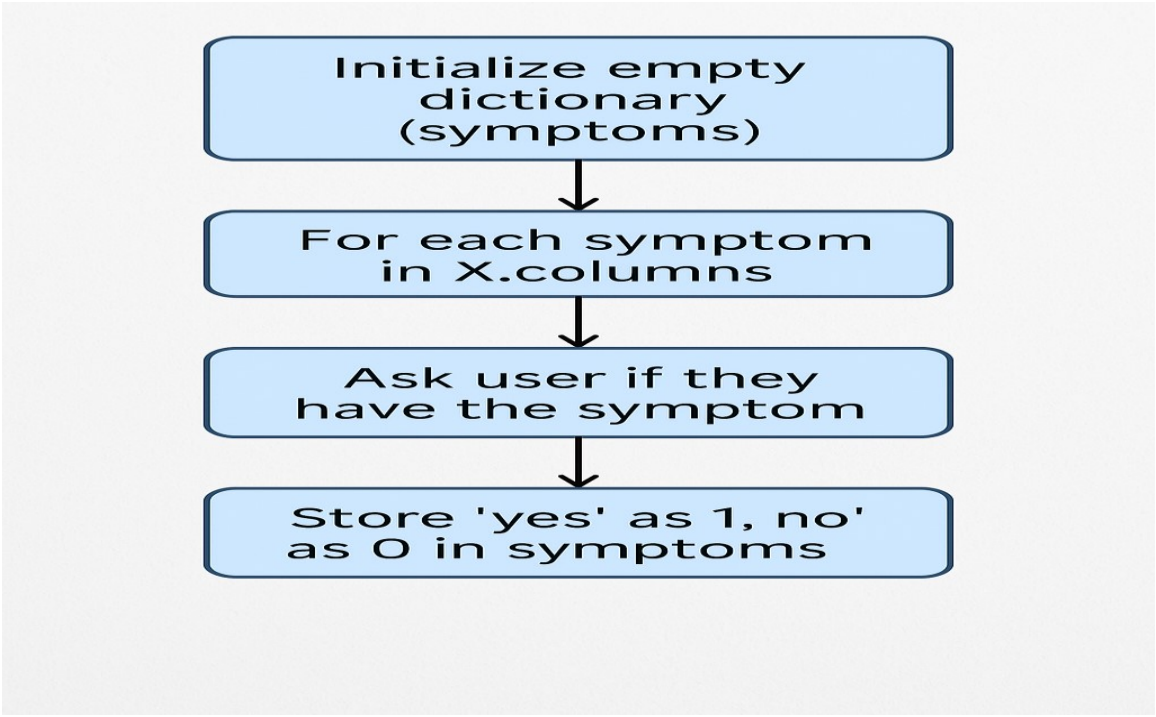
Step 4: Function to Ask Follow-Up Questions

This function takes a symptom, asks the user a question about it, and stores their yes/no answer.

Step 5: Main Interaction

Greet the user

- **# Collect symptoms**



- **#Predict disease**

  You're encoding the user's symptoms into a numerical format and passing them to the trained model to get a disease prediction.

- **# Display prediction**
- **# Recommend tests/medications**